Array creation functions

In [ ]:
```
# create an Array using list function
```

In [1]:
```
import numpy as np
a = np.array([1,2,3])
print("Array a:",a)
```

```
Array a: [1 2 3]
```

In [ ]:
```
#create an array with evenly spaced values
```

In [4]:
```
b = np.arange(0,10,2)  #values from 0 to 10 with step 2
print("Array b:",b)
```

```
Array b: [0 2 4 6 8]
```

In [6]:
```
#create an array filled with zeros
c=np.zeros((2,3))  # 2*3 array of zeros
print("Array c:\n",c)
```

```
Array c:
 [[0. 0. 0.]
 [0. 0. 0.]]
```

In [8]:
```
#create an array filled with ones
d=np.ones((3,2))
print("Array d:\n",d)
```

```
Array d:
 [[1. 1.]
 [1. 1.]
 [1. 1.]]
```

In [11]:
```
#create an array filled with ones
d=np.ones((3,2),dtype=int)  #it changes float to integer values inside matrix
print("Array d:\n",d)
```

```
Array d:
 [[1 1]
 [1 1]
 [1 1]]
```

In [ ]:
```
#create an identity matrix
#all diagonal values of a matrix u get values as 1
eg [[1 0 0]
    [0 1 0]
    [0 0 1]]
```

In [13]:
```
e=np.eye(4)
print("Array e:\n",e)
```

```
Array e:
 [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Array manipulation functions

In [16]:
```
# reshape an array
a1=np.array([1,2,3])
reshaped = np.reshape(a1,(1,3))
print("Reshaped array:", reshaped)
```

```
Reshaped array: [[1 2 3]]
```

In [17]:
```
# reshape an array
a1=np.array([1,2,3])
reshaped = np.reshape(a1,(3,1))
print("Reshaped array:", reshaped)
```

```
Reshaped array: [[1]
 [2]
 [3]]
```

In [20]:
```
# Flatten an array
#it is something changing all types of arrays to 1d array
#in the order mentioned
b1=np.array([[1,2],[3,2],[4,5],[6,7],[8,10]])
flattened = np.ravel(b1)
print("Flattened array:\n",flattened)
```

```
Flattened array:
 [ 1  2  3  2  4  5  6  7  8 10]
```

In [22]:
```
#Transpose an array
#its noting but changing the rows to columns and columns to rows
c1=np.array([[1,2],[4,7]])
Transpose = np.transpose(c1)
print("transpose array:\n",Transpose)
```

```
transpose array:
 [[1 4]
 [2 7]]
```

In [23]:
```
# Stack array vertically
a2 =np.array([1,2])
b2 =np.array([3,4])
stacked = np.vstack([a2,b2])
print("Stacked arrays:\n", stacked)
```

```
Stacked arrays:
 [[1 2]
 [3 4]]
```

In [24]:
```
# Stack array horizontal
a2 =np.array([1,2])
```

```
b2 =np.array([3,4])
stacked = np.hstack([a2,b2])
print("Stacked arrays:\n", stacked)
```

```
Stacked arrays:
 [1 2 3 4]
```

Mathematical functions

In [25]:
```
#ADD 2 arrays
f=  np.array([1,2,3,4])
added=np.add(f,2)  # we are adding 2 for each element
print("Addition of 2:",added)
```

```
Addition of 2: [3 4 5 6]
```

In [ ]:
```
# square each element
```

In [28]:
```
g=  np.array([1,2,3,4])
squared=np.power(g, 2)  # we are adding 2 for each element
print("square of 2:",squared) #it needs to be mentioned power of 3 or 4 or 5 i.e, e
```

```
square of 2: [ 1  4  9 16]
```

In [30]:
```
h=  np.array([1,2,3,4])
squared1=np.square(h)  # we are adding 2 for each element
print("square of 2:",squared1)
#it just works as only anything power of 2
```

```
square of 2: [ 1  4  9 16]
```

In [33]:
```
a3 = np.array([1,2,3])
dot_product = np.dot(a1, a3) #dot product of a and b
print(a1)
print(a3)
print("Dot product of a1 and a3:", dot_product) #it multiplies each other and addds
# when elements are same count like 3*3 and 3*3 but not with3*4
```

```
[1 2 3]
[1 2 3]
Dot product of a1 and a3: 14
```

Statistical funtion

In [34]:
```
s = np.array([1,2,3,4])
mean = np.mean(s)  #it is adding all the element s and didvind them with total coun
print("Mean of s:", mean)
```

```
Mean of s: 2.5
```

In [39]:
```
# variance of an array
var = np.var(s)
print("variance of s:", var)
```

```
variance of s: 1.25
```

In [35]:
```
# Standard deviation of an array
std_dev = np.std(s)
print("Standard Deviation of s:", std_dev)
```

```
Standard Deviation of s: 1.118033988749895
```

In [37]:
```
#minimum element of an array
minimum = np.min(s)
print("Min of s:", minimum)
```

```
Min of s: 1
```

In [38]:
```
#maximum element of an array
maximum = np.max(s)
print("Max of s:", maximum)
```

```
Max of s: 4
```

Random Sampling Functions

In [41]:
```
# Generate random values from 0 and 1
random_values = np.random.rand(3)  # Array for 3 random values btw 0 and 1
print(" Random values:",random_values)
```

```
 Random values: [0.24660204 0.96253153 0.75413654]
```

In [42]:
```
# Set seed reproductibility
np.random.seed(0)
#Generate random values btw 0 and 1
random_vals = np.random.rand(3) # Array of 3 random values btw 0 and 1
print("Random values as :",random_vals)  #it generates random values only once and
```

```
Random values as : [0.5488135  0.71518937 0.60276338]
```

In [45]:
```
# Generate random integers
random_intvals = np.random.randint(0, 10, size=5)
# Generate int values 0 and 1
print("Random int values as :",random_intvals)
```

```
Random int values as : [6 8 8 1 6]
```

In [47]:
```
# Set seed reproductibility
np.random.seed(0)
#Generate random int values btw 0 and 1
random_intvals1 = np.random.randint(0,10, size=5) # Array of 3 random values btw 0
print("Random values as :",random_intvals1)
```

```
Random values as : [5 0 3 3 7]
```

Boolean and Logical Functions

In [13]:
```
# Check if all elements are TRUE
# all
```

```
import numpy as np
logical_test = np.array([True, False, True])
all_True = np.all(logical_test)
print("All elements True:", all_True)
```

```
All elements True: False
```

In [14]:
```
# Check if all elements are TRUE
# all
import numpy as np
logical_test = np.array([True, True, True])
all_True = np.all(logical_test)
print("All elements True:", all_True)
```

```
All elements True: True
```

Set operations

In [15]:
```
# Intersection of 2 arrays
set_a = np.array([1,2,3,4])
set_b = np.array([3,4,5,6])
intersection = np.intersect1d(set_a,set_b)
print("Intersection of a and b:", intersection)
```

```
Intersection of a and b: [3 4]
```

In [16]:
```
#union of 2 arrays
union = np.union1d(set_a,set_b)
print("union of a and b:", union)
```

```
union of a and b: [1 2 3 4 5 6]
```

In [17]:
```
# Array Attributes
a = np.array([1,2,3])
shape =a.shape
size = a.size
dimensions = a.ndim
dtype = a.dtype
print("Shape of a:", shape)
print("size of a:", size)
print("dimensions of a:", dimensions)
print("dtype of a:", dtype)
```

```
Shape of a: (3,)
size of a: 3
dimensions of a: 1
dtype of a: int64
```

Other Functions

In [18]:
```
copied_array = np.copy(a)
print("Copied array is :", copied_array)
```

```
Copied array is : [1 2 3]
```

In [19]:
```
array_size_in_bytes = a.nbytes
print("size of bytes:", array_size_in_bytes)
```

```
size of bytes: 24
```

In [ ]:
```
# check if 2 arrays share memory
shared = np.shares_memory()
```

In [ ]:

In [ ]: